

World Cup 2026 Project Roadmap

Recommended next feature: Player detail pages

Why this is the best next step: your app already links to player detail routes, but those routes do not exist yet. Building them closes a real product gap while teaching routing, reusable data structures, dynamic pages, and component composition.

4-Week Roadmap

Week 1: Stabilize the app structure

Goal: make the project feel like one React app instead of part React and part static HTML.

- Move Cities, Stadiums, and Timeline into React routes instead of separate HTML pages.
- Replace plain anchor tags for internal navigation with Link or NavLink.
- Fix the countdown interval cleanup.
- Move hardcoded content into small data files where possible.
- Put EmailJS keys into environment variables instead of keeping them inline in the component.

Learning focus: React Router basics, useEffect cleanup, component vs data separation, and environment variables.

Definition of done: all main navigation works inside React, there are no broken internal links, and lint is clean.

Week 2: Build the player detail system

Goal: turn Players to Watch into a complete feature, not just a gallery.

- Create a players data file with each player's slug, image, country, club, position, and short profile.
- Add a dynamic route like /players/:slug.
- Build one reusable player detail page component.
- Show a not-found state for invalid slugs.
- Add a back link and optional next-player navigation.

Learning focus: route params, rendering from data, reusable page templates, and handling missing states.

Definition of done: every player card opens a working detail page, and adding a new player only requires updating the data file.

Week 3: Add one strong fan feature

Recommended pick: Favorites saved to localStorage.

Why: it is learnable, makes the app interactive, and teaches state, persistence, and user-centered product thinking.

- Let users favorite players.
- Persist favorites after refresh with localStorage.
- Add a simple favorites section or page.

Learning focus: local state vs persisted state, localStorage, and deriving UI from saved data.

Definition of done: favorites persist after refresh and are visible in the UI.

Week 4: Polish and ship

Goal: make the product presentable and reliable.

- Improve responsive layout, especially the slider and player cards.
- Add loading, success, and error states to the email form.
- Improve accessibility: button labels, alt text review, and keyboard navigation.
- Clean up text typos and naming consistency.
- Write a stronger README with features, stack, screenshots, and what you learned.
- Deploy the app and treat the deployed version as the real milestone.

Learning focus: responsive CSS, accessibility fundamentals, product polish, and deployment workflow.

Definition of done: the mobile experience is solid, the main features feel intentional, and the project is deployed and shareable.

How to use AI without letting it carry the project

- Use AI for explanations, debugging help, code review, and tradeoff analysis.
- Avoid asking AI to build whole features before you try.
- Do not keep code you cannot explain line by line.
- A good pattern: try first, get stuck, ask one narrow question, implement the fix yourself, then explain what changed.

Best build habit

For each feature: define the user outcome in one sentence, sketch the components, routes, and data you need, build the smallest version that works, test it manually, then ask AI for review or explanation after you have code.

Best next task

- Create a playersData file.
- Add a /players/:slug route.
- Build one PlayerDetail component.
- Make all player cards use that route.
- Add a not-found state.